PROJET ROBOTIQUE · RASPBERRY PI

PILOTAGE INTELLIGENT

Robot autonome d'évitement d'obstacles

Dossier technique complet · conception, montage, mise en service

Python | OpenCV | RPi.GPIO | HC-SR04 | L298N



INTRODUCTION

Présentation et objectifs

Le projet consiste à réaliser un **robot mobile autonome** capable de détecter et de contourner les obstacles en temps réel, sans intervention humaine.

1. **Percevoir** — mesurer la distance aux obstacles et repérer leur position
2. **Décider** — choisir le comportement adapté selon le niveau de risque
3. **Agir** — piloter les deux moteurs pour éviter sans s'arrêter

Contrainte : tout le traitement s'exécute en local sur Raspberry Pi, sans GPU ni connexion réseau.



Vue d'ensemble du robot terminé

VUE D'ENSEMBLE

Architecture logicielle



Boucle temps réel : perception → décision → action, exécutée en continu. Quatre modules découplés, testables un par un.

MATÉRIEL

Liste des composants

Composant	Rôle	Qté
Raspberry Pi 4/5	Cerveau : traitement et commande	**1**
HC-SR04	Capteur de distance (ultrasons)	**1**
Caméra USB	Détection latérale des obstacles	**1**
Driver L298N	Pont en H : commande des moteurs	**1**
Moteurs CC + roues	Propulsion (gauche / droite)	**2**
Batterie 7–12 V	Alimentation des moteurs	**1**
R 1 kΩ + 2 kΩ	Pont diviseur sur la ligne ECHO	**2**
Châssis, fils, breadboard	Structure et câblage	—

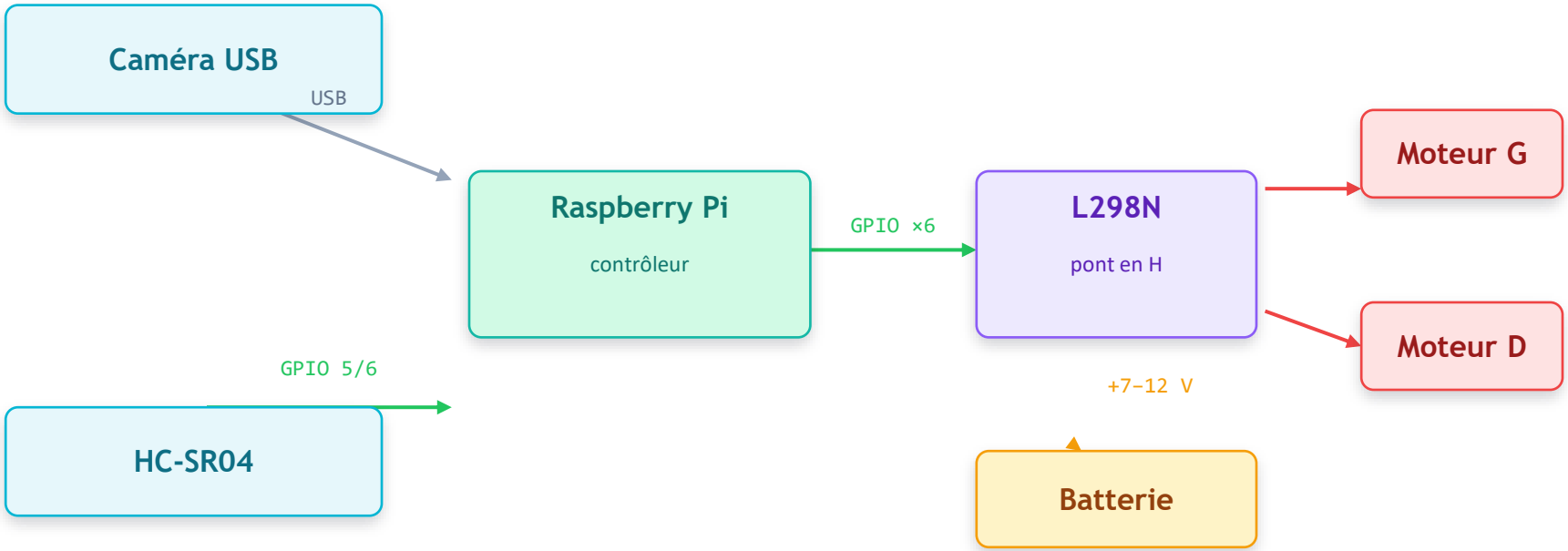


Insérez votre photo ici

Vos composants avant montage

MONTAGE

Schéma de câblage global



Astuce : reliez toutes les masses ensemble (Pi, L298N, batterie) — c'est l'erreur de câblage la plus fréquente.

MONTAGE

Brochage GPIO (numérotation BCM)

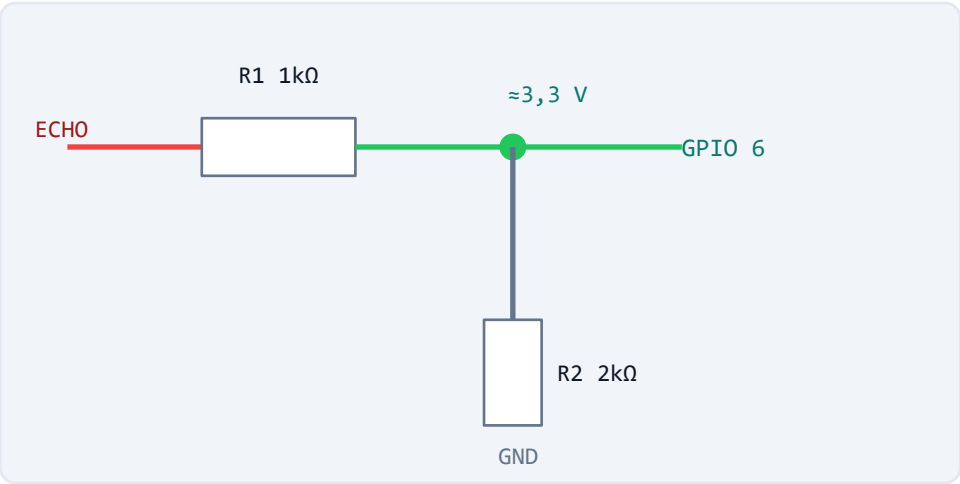
Broche Pi	Composant	Fonction
GPIO 5	HC-SR04 TRIG	Déclenche l'impulsion ultrason (direct)
GPIO 6	HC-SR04 ECHO	Retour de l'écho — via pont diviseur
GPIO 17 / 27	L298N IN1 / IN2	Sens du moteur gauche
GPIO 22 / 23	L298N IN3 / IN4	Sens du moteur droit
GPIO 18	L298N ENA	Vitesse PWM — moteur gauche
GPIO 24	L298N ENB	Vitesse PWM — moteur droit
5 V / GND	HC-SR04 / L298N	Alimentation logique et masse commune

Seule la ligne ECHO nécessite une adaptation de tension : elle sort du 5 V, or les entrées du Pi tolèrent 3,3 V maximum.

MONTAGE · POINT DÉLICAT

Pont diviseur sur la ligne ECHO

Pourquoi : ECHO sort un signal 5 V, incompatible avec les entrées 3,3 V du Pi. Deux résistances abaissent la tension.



$$V = 5 \times R2 / (R1 + R2) = 5 \times 2/3 \approx 3{,}3\ V$$

Insérez votre photo ici

Gros plan du pont diviseur sur la breadboard

ACTIONNEURS · PRINCIPE

Le driver L298N : un double pont en H

Quatre interrupteurs entourent chaque moteur. Fermer une paire diagonale fait circuler le courant dans un sens ; l'autre diagonale l'inverse.



Direction

Les entrées IN fixent le sens : IN1=H,IN2=L → avant ; inversé → arrière.



Vitesse

Un signal PWM sur ENA/ENB règle la vitesse (rapport cyclique = % de vitesse).



Puissance

Les moteurs sont alimentés par la batterie via le L298N, jamais par le Pi.

Bon à savoir : le L298N chauffe et « perd » 1,5–2 V ; ses diodes de protection sont déjà intégrées.

CODE · ULTRASONIC.PY

Mesure de distance par temps de vol

```python
def distance_cm(self):
    GPIO.output(TRIG, GPIO.HIGH)
    time.sleep(0.00001)   # 10 us
    GPIO.output(TRIG, GPIO.LOW)
    while GPIO.input(ECHO) == 0:
        start = time.time()
    while GPIO.input(ECHO) == 1:
        end = time.time()
    delta = end - start
    return delta * 34300 / 2  # cm
```

Principe

- Impulsion TRIG de 10 µs
- Mesure de la durée de l'écho
- Distance = durée × 343 m/s ÷ 2
- Timeout 30 ms anti-blocage

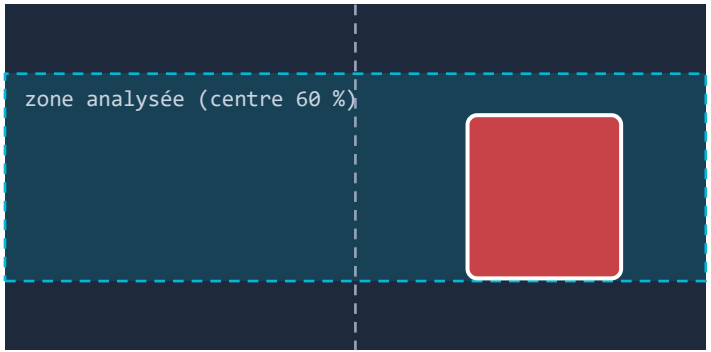
Portée ~5 m · zone aveugle < 2 cm

CODE · DETECTOR.PY

Détection d'obstacle (OpenCV)

```python
# soustraction de fond adaptative (MOG2)
mask = self._bg.apply(band)
contours, _ = cv2.findContours(mask,
    cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
for cnt in contours:      # plus gros objet
    if cv2.contourArea(cnt) < MIN_AREA:
        continue
    cx = int(M['m10'] / M['m00'])
# biais : -1 gauche, +1 droite
bias = (cx - FRAME_W/2) / (FRAME_W/2)
```

Sortie : un biais latéral dans [-1 ; +1]



L'obstacle le plus proche donne la direction à éviter ; combiné à la distance, il pilote le braquage.

CODE · MOTORS.PY

Contrôle différentiel via L298N

```
def _set_motor(self, in1, in2, pwm, speed):
    speed = max(-100, min(100, speed))
    if speed >= 0:        # marche avant
        GPIO.output(in1, GPIO.HIGH)
        GPIO.output(in2, GPIO.LOW)
    else:                 # marche arriere
        GPIO.output(in1, GPIO.LOW)
        GPIO.output(in2, GPIO.HIGH)
    pwm.ChangeDutyCycle(abs(speed))
```

Signe de la vitesse → sens de rotation. Valeur absolue → rapport cyclique PWM (la vitesse).

`set(left, right)` applique une vitesse à chaque roue : c'est la différence des deux qui fait tourner le robot.

LOGIQUE · MAIN.PY

Décision selon la distance : 3 zones



DANGER

`distance < 15 cm`

Stop, marche arrière 0,5 s puis demi-tour à l'opposé de l'obstacle



ATTENTION

`distance < 50 cm`

Évitement progressif : la caméra donne le biais, braquage proportionnel



LIBRE

`distance ≥ 50 cm`

Marche avant en ligne droite à la vitesse de base (60 %)

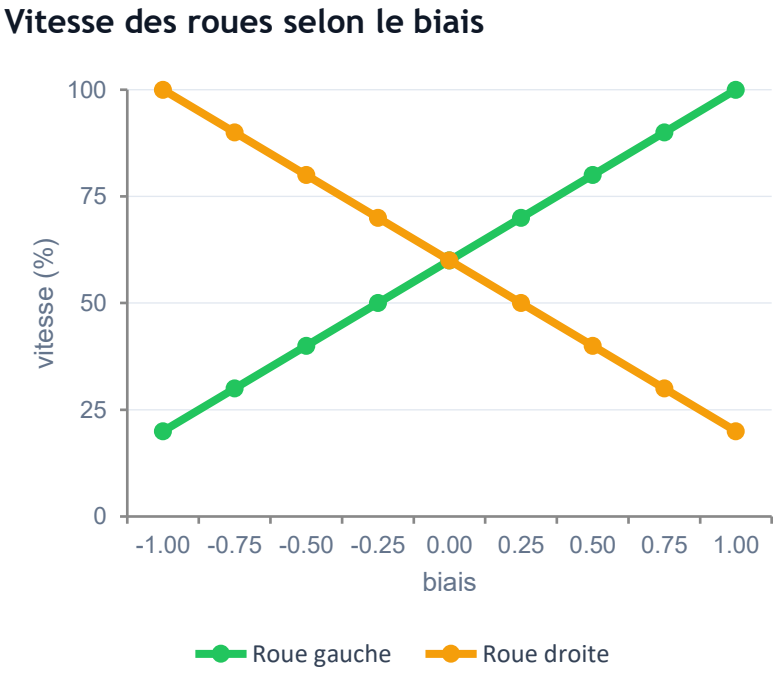
« Plus l'obstacle est proche, plus la réaction est forte. »

CŒUR DU CONTRÔLE · MAIN.PY

Braquage proportionnel

```
def compute_wheel_speeds(bias, distance):
    proximity = (WARNING_CM - distance) / \
                (WARNING_CM - DANGER_CM)
    delta = bias * TURN_FACTOR * proximity
    left  = BASE_SPEED + delta
    right = BASE_SPEED - delta
    return left, right
```

delta combine **direction** (biais) et **urgence** (proximité). Les roues reçoivent des vitesses opposées : le robot pivote vers le côté libre.



MISE EN SERVICE

Tester chaque élément avant la boucle

Avant de lancer main.py, on vérifie capteur et moteurs séparément. Roues en l'air pour le test moteurs.

`test_ultrasonic.py`
Affiche la distance en continu : la valeur doit suivre la main.

`test_motors.py`
Fait tourner chaque moteur avant/arrière : vérifie sens et côté.

Sens inversé ? Échangez les 2 fils de sortie du moteur sur le L298N — aucun code à modifier.

Insérez votre photo ici

Le robot surélevé en phase de test

MISE EN SERVICE

Lancement et dépannage

Lancement

```
cd robot
python3 main.py

# arret propre : Ctrl + C
```

Sur Pi sans écran (SSH), mettre SHOW_VIDEO = False dans main.py.

Dépannage courant

- **« SOC peripheral base address »** — RPi.GPIO incompatible (Pi 5 / Bookworm) → installer rpi-lgpio
- **Capteur renvoie l'infini** — Vérifier le pont diviseur, le 5 V et la masse commune
- **Moteur à l'envers** — Inverser ses 2 fils de sortie sur le L298N
- **Moteurs erratiques** — Masse commune manquante entre Pi, L298N et batterie

CONCLUSION

Synthèse et perspectives

CE QUI FONCTIONNE

- Fusion ultrason (distance) + caméra (direction)
- Réaction proportionnelle à la proximité
- Architecture modulaire et testable
- Temps réel sur Raspberry Pi, sans GPU

PISTES D'AMÉLIORATION

- **Régulation PID** — lisser le braquage, éviter les oscillations
- **Détection par ML** — YOLO embarqué à la place du fond
- **Multi-capteurs** — plusieurs HC-SR04 pour élargir le champ
- **Filtrage** — moyennage des mesures du capteur

Espace pour une photo finale du robot en action